Reproducibility Certification

Reproducibility Certification I

This section provides a machine-verifiable reproducibility certificate for the complete study. Every metric below is computed by the analysis pipeline and injected into the manuscript at render time — establishing a cryptographic chain of custody from configuration to publication.

Configuration Provenance

Configuration Provenance I

Property	Value
Config hash (SHA-256, truncated)	`eab2b6c6d272dc27`
Paper version	3.0.0
First author	Daniel Ari Friedman
Keywords	optimization algorithms, gradient descent, convergence analysis, numerical methods, mathematical programming, reproducible research, infrastructure automation

Configuration Provenance I

The configuration hash changes whenever any parameter in `config.yaml` is modified, ensuring that every rendered PDF is traceable to a specific configuration state.

Generated Artifact Registry

Generated Artifact Registry I

The analysis pipeline produced the following artifacts, each validated by `infrastructure.validation.output.validator`:

Generated Artifact Registry I

Category	Count
Publication-quality figures	7
Structured data files (CSV/JSON)	3
Analysis reports	6
Total artifacts	**16**

Numerical Validation Summary

Convergence Verification I

Within the configured grid, **4** of **6** runs satisfied `gradient_descent()` convergence (`No` indicates whether every row in `optimization_results.csv` converged).

Convergence Verification I

- Converged step sizes: 0.1, 0.5, 1.0, 1.5
- Non-convergent or hit-iteration-cap step sizes: 0.01, 2.5
- Smallest recorded iteration count: 1 (at $\alpha = 1.0$)
- Largest recorded iteration count: 1000 (at $\alpha = 0.01$)
- Mean iterations across all rows: 351

Numerical Stability I

Stability score from `infrastructure.scientific.stability`: **1.00** (out of 1.00)

The stability analysis tested 48 parameter combinations (8 starting points $\times$ 6 step sizes), recording termination accuracy across this finite gradient-descent grid. This is distinct from the reported stability score, which checks scalar objective evaluations and does not certify convergence of every optimization trajectory.

Benchmark Demonstration I

This exemplar also demonstrates `infrastructure.benchmark`. The thin orchestrator `scripts/04_benchmark_stage.py` calls `src/template_code_project/core/benchmark_support.py`, which evaluates the pure `quadratic_function` across fixed seeded inputs and turns reproducible completion, finiteness, and output-stability facts into boolean rubric checks. Those checks are scored through `infrastructure.benchmark.score_rubric` against a weighted `RubricSet`, then rendered with `scores_to_markdown` into the byte-stable `output/reports/benchmark_report.json` with a deterministic objective-value figure. Real timing still executes as a runtime diagnostic, but it is not serialized into tracked evidence.

Madlib Injection Verification

Madlib Injection Verification I

This manuscript demonstrates the template's "madlib" capability: every quantitative claim is injected from computed data at render time. The substitution system processed the following variables:

Madlib Injection Verification I

- **Configuration variables**: Drawn from `manuscript/config.yaml` (`experiment:` section)
- **Result variables**: Computed from `output/data/optimization_results.csv`
- **Stability variables**: Extracted from `output/reports/stability_analysis.json`
- **Provenance variables**: Generated at substitution time (timestamps, hashes, versions)

Madlib Injection Verification I

To verify: modifying any value in `config.yaml` and re-running the pipeline will automatically update every corresponding claim in this document. No manual transcription is required or permitted.